Laboratorio 4: El Validador de Perfiles (Null Safety en UI)

1. Introducción

En la ingeniería de software móvil, la estabilidad de una aplicación depende de cómo manejamos la ausencia de datos. Mientras que en Java esto solía causar el cierre inesperado de la App (*NullPointerException*), en **Kotlin Moderno** la seguridad es un contrato garantizado por el compilador.

2. Objetivos de Aprendizaje

- Implementar el sistema de tipos de Kotlin para manejar datos nulos simulados de una base de datos.
- Utilizar el operador **Elvis (`?:`)** y **Safe Calls (`?.`)** para prevenir errores en la interfaz de usuario.
- Aplicar la estructura `when` como una expresión funcional para determinar estilos visuales dinámicos.

3. Requisitos previos

- Haber completado el **Laboratorio 1** (Estandarización) y **Laboratorio 2** (Primera App).
- Contar con el emulador **Pixel 8 (API 36)** operativo.
- Tener configurada la identidad de **Git** y cuenta en **GitHub**.

4. Descripción general

Los estudiantes crearán una pantalla de perfil de usuario. Los datos (Avatar, Biografía y Seguidores) vendrán de variables que pueden ser nulas. La aplicación debe ser "indestructible": si los datos faltan, la interfaz debe mostrar valores por defecto elegantes sin cerrarse.

5. Paso a paso detallado

Paso 1: Clonación y Preparación

Para mantener las versiones de la API 36 y el bloqueo de Gradle del Lab 1, "clonaremos" el proyecto anterior.

1. En el explorador de archivos de Windows, **copiar** la carpeta de tu `RDA1_Laboratorio2_NombreApellido`.
2. **Pegarla** en su carpeta de trabajo y cambiarle el nombre a: `RDA1_Laboratorio4_NullSafety_NombreApellido`.

3. **Limpiar el caché:** Borrar las carpetas ocultas .gradle y build dentro de la nueva carpeta.
4. **Abrir Android Studio Ladybug** y selecciona "Open" para buscar tu nueva carpeta.

Paso 2: Renombrado del Proyecto y Sincronización

1. En el panel izquierdo, desplegar **Gradle Scripts** y abrir el archivo settings.gradle.kts.
2. **Reemplazar** el nombre actual por: rootProject.name = "ProfileValidator".
3. **Sincronizar:** Hacer clic en el icono del **Elefante (Sync Project with Gradle Files)** en la esquina superior derecha.
4. **Verificación:** Abrir la **Terminal** (inferior izquierda) y ejecutar ./gradlew --version. Debe decir **JVM: 21**.

Paso 3: Configuración de la "Base de Datos" Simulada

Trabajaremos en el archivo MainActivity.kt. Para evitar errores de visibilidad, definiremos los datos como variables globales (fuera de la clase).

1. Navegar a: app/src/main/java/[tu.paquete]/MainActivity.kt.
2. **Borrar todo el contenido** del archivo, excepto la primera línea (el package).
3. **Pegar las importaciones necesarias**

```
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.*
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.AccountCircle
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.unit.dp
```

4. **Agregar las variables de datos simulados** justo después de las importaciones (fuera de cualquier clase):

```
// Datos simulados (Variables de nivel superior para acceso global)
val avatarUrl: String? = null
val userBio: String? = "Desarrollador Android en formación."
val followerCount: Int? = 1500
```

Paso 4: Implementación de la Clase y UI Segura (Jetpack Compose)

1. **Declarar la clase MainActivity** debajo de las variables:

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MaterialTheme {
                Surface(
```

```
                modifier = Modifier.fillMaxSize(),
                color = MaterialTheme.colorScheme.background
            ) {
                UserProfileScreen()
            }
        }
    }
}
```

Explicación:

- **onCreate**: Es el "punto de entrada" de su pantalla. Imagine que es el main de su aplicación móvil. Se ejecuta cuando la actividad se crea por primera vez.
- **setContent**: Es la instrucción que le dice a Android: "Deja de usar archivos XML antiguos y empieza a dibujar la interfaz usando Jetpack Compose".
- **MaterialTheme y Surface**: Son los cimientos visuales. El primero aplica los estilos (colores, tipografía) de Google, y el segundo prepara el lienzo (fondo) donde dibujaremos.

2. **Añadir la función Composable** al final del archivo (fuera de la clase):

```
@Composable
fun UserProfileScreen() {
    // 1. Manejo seguro de nulidad con el Operador Elvis (?:)
    val bioDisplay = userBio ?: "Biografía no disponible"

    // 2. 'when' como expresión para lógica de colores
    val statusColor = when {
        followerCount == null -> Color.Gray
        followerCount > 1000 -> Color(0xFFFFD700) // Dorado
        followerCount > 100 -> Color.Blue
        else -> Color.DarkGray
    }

    Column(
        modifier = Modifier.fillMaxSize().padding(24.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        // Icono genérico con color dinámico basado en nulidad
        Icon(
            imageVector = Icons.Default.AccountCircle,
            contentDescription = null,
            modifier = Modifier.size(100.dp),
            tint = if (avatarUrl == null) Color.LightGray else Color.Green,
        )

        Spacer(modifier = Modifier.height(16.dp))
```

```
        Text(text = bioDisplay, style = MaterialTheme.typography.bodyLarge)

        // Llamada segura (?.) para mostrar el número de seguidores
        Text(
            text = "Seguidores: ${followerCount ?: 0}",
            color = statusColor,
            style = MaterialTheme.typography.headlineSmall,
        )
    }
}
```

Explicación:

- **@Composable**: Esta anotación le indica a Kotlin que esta función no es una función normal, sino una "pieza de interfaz". Es como un bloque de LEGO que puede reutilizar.
- **Column**: Es un contenedor que organiza todo lo que ponga dentro uno debajo de otro (verticalmente).
- **Lógica de UI**: Fijarse que estamos usando when e if directamente mientras dibujamos. En Android moderno, la lógica y la interfaz viven juntas de forma segura.

Paso 5: Ejecución y Pruebas

1. **Ejecutar la aplicación**: Presionar el botón Run (flecha verde) o Shift + F10.
2. **Verificación en el Emulador**: Debe ver el icono gris, la biografía escrita y el número de seguidores en dorado.
3. **Prueba de Robustez**: Cambiar val followerCount: Int? = 1500 por null en el código, vuelva a ejecutar y observar cómo la App muestra "0" y color Gris sin cerrarse:

6. Resultados esperados

- El proyecto compila sin errores de "Nullability mismatch".
- La UI se adapta dinámicamente: si cambia `followerCount` a `50`, el texto debe ponerse gris automáticamente al reiniciar.
- **Cero cierres (Crashes):** Aunque todas las variables se pongan en `null`, la App debe mostrar "Biografía no disponible" y "Seguidores: 0".

7. Desafío extra: El "Filtro de Seguridad"

Reto: Modificar el código para que, si el usuario intenta ingresar un número de seguidores negativo o un texto inválido, la App asigne `0` automáticamente usando `.toIntOrNull()`.

8. Preguntas de Reflexión

- **Seguridad frente a Java:** ¿Por qué cree que manejar la nulidad en el compilador (antes de ejecutar) es mejor que manejarla con bloques try-catch o if(x != null) en Java?
- **Operador Elvis:** En el código userBio ?: "Biografía no disponible", ¿qué sucedería con la interfaz de usuario si no usáramos este operador y la variable fuera nula?
- **Flexibilidad de when:** ¿Cómo facilita el uso de when como expresión la legibilidad del código al asignar colores dinámicos en comparación con múltiples sentencias if-else?
- **Experiencia de Usuario (UX):** ¿Qué impacto tiene en la percepción del usuario que la aplicación muestre un valor por defecto (como "0" seguidores) en lugar de cerrarse inesperadamente?
- **Inmutabilidad:** ¿Por qué se prefiere usar val en lugar de var para los datos que vienen de la "base de datos" simulada en esta práctica?

9. Entregables

Deberá subir al EVA un archivo **PDF** con:

1. Informe que la estructura básica de siempre no olvide de responder a las preguntas planteadas.
2. Captura de pantalla de la **Terminal** con la salida exitosa de `./gradlew --version`.
3. Captura del **Emulador API 36** mostrando la UI con los tres estados de nulidad manejados.
4. Captura del **Git Log** en Android Studio mostrando el commit: `Lab3: Manejo de Null Safety en Perfil de Usuario`.
5. Enlace (URL) público de tu repositorio de **GitHub**.

9. Rúbrica de Evaluación

- No olvide revisar la rúbrica de evaluación, en el EVA, para complementar su informe.